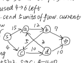
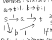
1.

1. Augmenting path 1: path: $s \to a \to b \to t$
Capacities on this path: $s \to a$: 15, $a \to b$: 12, $b \to t$: 7. bottleneck = min (15, 12, 7) = 7. send 7 units of flow, current total flow: 7
Augmenting path 2: path: $s \to c \to d \to t$
Capacities remaining on these edges: $s \to c$: 4, $c \to d$: 10, $d \to t$: 10
bottleneck = min (4, 10, 10) = 4. send 4 units of flow, current total flow = 7+4 = 11
Augmenting path 3: path: $s \to a \to b \to c \to d \to t$
Remaining capacities on these edges now: $s \to a$: was 15, used 7 → 8 left, $a \to b$: was 12, used 7 → 5 left, $b \to c$: 5 (unused so far). $c \to d$: was 10, used 4 → 6 left, $d \to t$: was 10, used 4 → 6 left
bottleneck = min (8, 5, 5, 6, 6) = 5. send 5 units of flow. current total flow = 11+5 = 16



Check that we are at maximum
After these 3 augmenting paths, the residual capacities on outgoing edges from s are: $s \to a$: 15 − (7+5) = 3, $s \to c$: 4 − 4 = 0
From a, the only forward edge $a \to b$ is now saturated 12 − (7+5) = 0
So from s we can only go to a, but from a we cannot go forward to the rest of the graph (only backward edges exit). Thus, no more s-t augmenting path exists.
We can also see a minimum cut: Take S = {s, a}, T = {b, c, d, t}, Cut edges S → T are: $s \to c$ (4) and $a \to b$ (12). cut capacity = 4+12 = 16
∴ Maximum flow value = 16.

2. False. Counterexample: Consider this small flow network: vertices: s, a, b, t, directed edges and capacities: $s \to a$: 1, $s \to b$: 1, $a \to t$: 1, $b \to t$: 1



① Compute the maximum flow: There are clearly 2 disjoint s-t paths: $s \to a \to t$ (capacity 1), $s \to b \to t$ (capacity 1). So the maximum flow is: $|f^*| = 1+1 = 2$
② Find minimum cuts: Loot at the cut (S, T) with S = {s, a}, T = {b, t}. edges from S to T are: $s \to b$ with capacity 1, $a \to t$ with capacity 1 so the capacity of this cut is: C (S, T) = 1+1 = 2. This equals the maximum flow, so (S, T) is a minimum cut.
③ Increase the capacity of an edge in the min cut
Take the minimum cut we just found: S = {s, a}, T = {b, t}, cut edges: $s \to b$ and $a \to t$.
Pick edge $a \to t$ (which is in the cut) and increase its capacity from 1 to 100. Now capacities are: $s \to a$: 1, $s \to b$: 1, $a \to t$: 100, $b \to t$: 1.
Look at the source-side edges: Total capacity out of s is still 1+1 = 2. So the maximum flow from s to t cannot exceed 2, no matter how large $a \to t$ is. In fact, we can still send at most: 1 unit along $s \to a \to t$, 1 unit along $s \to b \to t$. So the new maximum flow is still $|f^*_{new}| = 2$. Therefore, the statement is false.

3. We can use a max-flow reduction with unit capacities.
① Build a flow network: Construct a flow network G′ = (V, E) from G as follows: For every directed edge (u, v) ∈ E, create the same edge in G′ with capacity 1: C (u, v) = 1 for all (u, v) ∈ E. Use the same source s and sink t.
② Run a max-flow algorithm. Run any standard max-flow algorithm (Edmond-Karp or Dinic) on G′ from s to t. Let the value of the maximum flow be F. Because all capacities are integers (all 1), the max-flow algorithm returns an integral flow (each edge has flow 0 or 1)
③ Extract the edge-disjoint paths
From the resulting flow: (1) Consider only edges with flow value 1. (2) Repeatedly: start from s, follow edges with flow 1 until you reach t, record the resulting path as one s-t path, remove all edges of that path from consideration (or mark them used). (3) Each such path uses only edges with flow 1, and since no edge has flow more than 1, these paths are edge-disjoint. (4) Continue until no s-t path remains in the "flow graph".
The number of paths we obtain is exactly F.
Running time: If we use Edmonds-Karp, $O(VE^2)$, or Dinic's algorithm: $O(E\sqrt{V})$ for unit-capacity graphs, then overall the algorithm runs in polynimal time in |V| and |E|.

4. let G = (V, E) be a directed graph with designated vertices s and t. ① Build a unit-capacity flow network: We turn G into a flow network G′ = (V, E, c) by assigning c(e) = 1 for every edge e ∈ E. We keep the same source s and sink t.
② Relate edge-disjoint paths to flows
From Question 3 we already know: Claim 1: In this unity-capacity network, the value of a maximum s-t flow is exactly equal to the maximum number of edge-disjoint s-t paths in G. Hence, the maximum flow value equals the maximum number of edge-disjoint s-t paths.
③ Relate separating edge sets to cuts
Recall the definition in the problem: A set F ⊆ E of edges separates s from t, if, after removing all edges in F, there is no s-t path left. We now connect such sets F to s-t cuts in the flow network.
An s-t cut (S, T) is a partition of V such that: s ∈ S, t ∈ T, and all edges from S to T are the cut edges.
The capacity of the cut is $c(S,T) = \sum_{(u,v) \in E,\, u \in S,\, v \in T} C(u,v)$
In our unit-capacity network, each edge from S to T has capacity 1, so: C(S, T) = # {edges from S to T}
Now: Removing all edges from S to T will certainly destroy all paths from s (in S) to t (in T). So the set of edges from S to T is a separating set. Conversely, any minimal separating set of edges corresponds to some cut: intuitively, put in S all vertices reachable from s when that separating set is removed; everything else in T. Then all remaining edges from S to T must be in the separating set.
Thus: Claim 2: In the unit-capacity network, the minimum number of edges whose removal separates s from t equals the minimum capacity of an s-t cut.
④ Apply the Max-Flow Min-Cut Theorem
The Max-Flow Min-Cut Theorem says: maximum flow value = minimum s-t cut capacity
Combining with claim 1 and claim 2: Maximum flow value = maximum number of edge-disjoint s-t paths. Minimum cut capacity = minimum number of edges whose removal separates s from t.
Therefore, maximum number of edge-disjoint s-t paths = minimum number of edges whose removal separates s from t.

5. The last digit of any integer is the value of the integer modulo 10. Since 17 ≡ 7 (mod 10). We can rewrite the problem as finding: $7^{17^{17}}$ (mod 10). Thus, we focus on the powers of 7 modulo 10.
The last digit of powers of 7 cycles every 4.

Power	last digit
7^1	7
7^2	9
7^3	3
7^4	1

Since $7^5 = 7^4 \cdot 7 = 7$
∴ cycle length = 4

Since the last digit repeats with period 4, we need 17^{17} mod 4. Notice: 17 ≡ 1 (mod 4). Using the rule that congruence is presened under exponentiation: $17^{17} \equiv 1^{17} \equiv 1$ (mod 4). Thus, the exponent 17^{17} is congruent to 1 modulo 4.
The cycle:

exponent mod 4	last digit
1	7
2	9
3	3
4	1

Since $17^{17} \equiv 1$ (mod 4), we choose the first element of the cycle. Therefore: $7^{17^{17}} \equiv 7$ (mod 10)
∴ The least significant decimal digit of $17^{17^{17}}$ is 7.

6. First compute Euler's totient: $\varphi(n) = (p-1)\cdot(q-1) = 10 \cdot 28 = 280$. We need d such that ed ≡ 1 (mod φ(n)) → 3d ≡ 1 (mod 280) Use the extended Euclidean algorithm: 280 = 3×93+1 ∴ 1 = 280 − 3×93 ∴ −93 is an inverse of 3 modulo 280. We can add 280 to get a positive representative. d = 280 − 93 = 187 ∴ d = 187. Encryption is $C = M^e$ (mod n) = 100^3 (mod 319) ① 100^2 = 10000, Reduce modulo 319: 319 × 31 = 9889 10000 − 9889 = 111 ∴ $100^2 \equiv 111$ (mod 319) ② $100^3 = 100^2 \cdot 100 \equiv 111 \cdot 100 = 11100$ (mod 319) Reduce 11100 modulo 319: 319 × 34 = 10846, 11100 − 10846 = 254 ∴ $100^3 \equiv 254$ (mod 319)
∴ the ciphertext is C = 254
Secret exponent d = 187. encryption of M = 100, C = 254

7. (a) IS-DECIDE ∈ NP
Given a candidate set V′, we can verify in O(|E|) time that no edge in E has both endpoint in V′. Thus IS-DECIDE is in NP.
NP-hardness (Reduction from CLIQUE)
Given an instance (G, k) of CLIQUE, construct the complement graph $\bar{G}$:
same vertex set
(u, v) ∈ E($\bar{G}$) if (u, v) ∉ E(G)
Then: G contains a clique of size k ⟺ $\bar{G}$ contains an independent set of size k.
Thus (G, k) ∈ CLIQUE if ($\bar{G}$, k) ∈ IS-DECIDE.
The reduction is polynomial time.
Therefore, IS-DECIDE is NP-complete.
(b) Using the decision oracle to compute a maximum independent set
Assume we have an oracle: IS-DECIDE (G, k) that returns TRUE/FALSE.
step 1: Determine the maximum size k*
Binary-search on k ∈ [0, |V|] to find the largest k* such that IS-DECIDE (G, k*) = TRUE
step 2: Construct the actual independent set
Initialize S = ∅, G_0 = G, and k = k*.
Process vertices one by one:
For each vertex v ∈ V:
1. Query the oracle on G_0 − {v}: IS-DECIDE (G_0 − {v}, k)
2. If the answer is FALSE, then every independent set of size k in G_0 must include v. Then:
Add v to S
Remove v and all neighbours of v from G_0.
Decrease k := k − 1
We finished, S is a maximum independent set.
Running time: Binary search uses O(log |V|) oracle calls, and the construction uses O(|V|) calls. Counting each oracle call as one unit, the algorithm runs in polynomial time.
(c) Polynomial-time algorithm when every vertex has degree ≤ 2
If every vertex has degree at most 2, the graph is a disjoint union of: paths and cycles. We solve each connected component separately using dynamic programming
Case 1: Path graph
Let the path be $v_1, v_2, \dots v_n$.
Define:
dp[i][0]: maximum independent-set size in $v_1 \sim v_i$ without v_i
dp[i][1]: same, including v_i
Transitions: dp[i][0] = max (dp[i−1][0], dp[i−1][1])
dp[i][1] = dp[i−1][0] + 1
Answer for the path: max (dp[n][0], dp[n][1])
Case 2: cycle path
let the cycle be $v_1, v_2, \dots v_n, v_1$, two possibilities:
1. Do not include v_1 → run the path DP on $v_2 \sim v_n$.
2. Include v_1 → then v_2 cannot be chosen → run path DP on $v_3 \sim v_n$, then add 1 for v_1.
Take the maximum of the 2 outcomes.
Correctness: Since each component is a path or a cycle and DP explores all valid "include/exclude" options respecting adjacency constraints, the DP produces an optimal independent sent for every component. Combining all components yields a global optimal solution.
Running time: Each component is processed in linear time. Overall complexity O(|V| + |E|). Thus, for graphs with maximum degree 2, the independent-set problem is polynomial-time solvable.